

remote-cluster-provisioner

Controllers User Guide

- RemoteCluster
- NodeProvision
- NodeProvisionNetConfig

Audience: this guide is for people who **run** `remote-cluster-provisioner` to create and manage remote GPU/CPU clusters — not for people building the project from source. It assumes the controller container image has already been built and is available in a registry your cluster can pull from (your platform team's job, or a pre-built image you were handed). Nothing here requires compiling Go code, editing source files, or building images — only `kubectl` and the manifests shipped in this repository.

1. Overview

1.1 What the controllers are

`remote-cluster-provisioner` runs as a single controller Deployment that manages three kinds of resources (CRDs), across two areas of responsibility:

Resource (Kind)	Where you create it	What it's for
RemoteCluster	Management cluster	Bootstraps a brand-new remote cluster (control plane + workers) from bare hosts
NodeProvision	Remote (workload) cluster	Adds worker nodes to a cluster that already exists
NodeProvisionNetConfig	Remote (workload) cluster	Shared cluster-wide configuration (VPN, join command, registry credentials) that <code>NodeProvision</code> reads from

The same controller image runs on both the management cluster and every remote cluster it creates — which behavior you get is simply a matter of which of these resources you create on which cluster.

`NodeProvisionNetConfig` is **not** something you interact with much directly: it's created and kept up to date automatically, and its only job is to hold shared settings (VPN range, join command, registry credentials) that `NodeProvision` reads when adding a node. You mainly touch it when you want to change a cluster-wide setting (e.g. rotate registry credentials) without editing every node individually.

1.2 What each controller does

Cluster bootstrap (RemoteCluster , applied on the management cluster) — given SSH access to a bare Ubuntu 22.04 host and a RemoteCluster resource, it turns that host into either:

- a fully working Kubernetes **control-plane** node (Kubernetes itself, container runtime, networking, and a GitOps platform stack), or
- a **worker** node joined to a control-plane you already created in the same logical cluster.

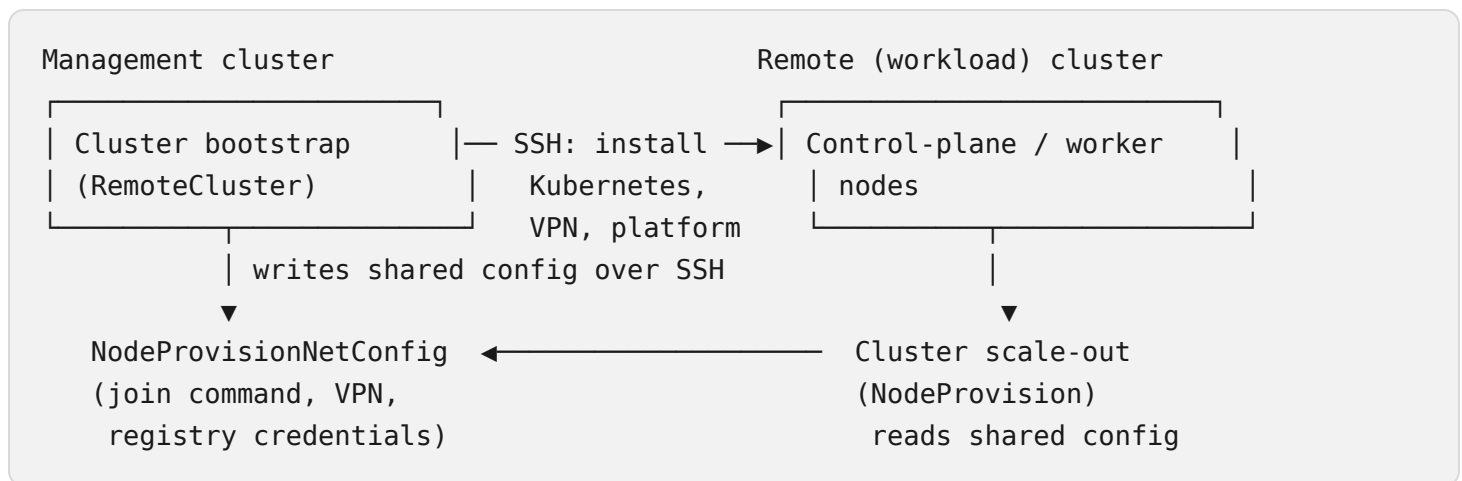
Once a cluster is up, it also keeps that cluster's shared configuration in sync (VPN settings, software versions, registry credentials), rotates its join token automatically, and cleanly tears a node back down (removes it from Kubernetes, from the VPN, and resets it) when you delete the RemoteCluster resource.

Cluster scale-out (NodeProvision , applied on the remote/workload cluster itself) — given a NodeProvision resource, it adds **one more worker node** to a cluster that a RemoteCluster has already bootstrapped, using either:

- **OnPrem** — an existing bare-metal/VM host you already have, reached over SSH, or
- **AWS** — a brand-new EC2 instance it launches for you (it can pick a sensible instance type, AMI, and network config automatically if you don't specify one).

This one keeps working even if the management cluster becomes unreachable — it can add nodes to its own cluster independently.

1.3 How they interact with each other



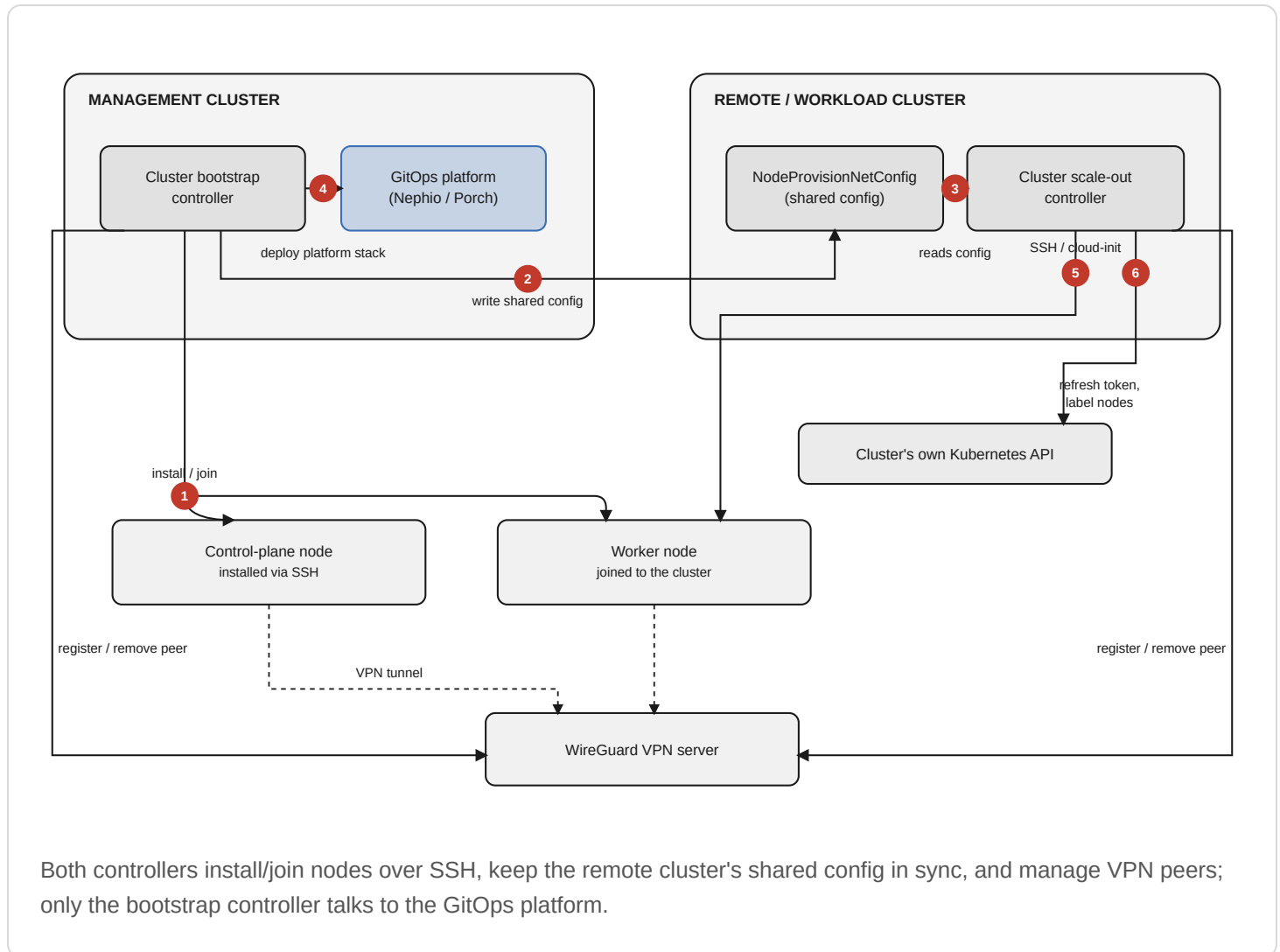
Cluster bootstrap is the only one of the two that talks to your GitOps platform (Nephio/Porch) and the only one that does the very first Kubernetes install on a control-plane node. Cluster scale-out never does that first install — it only ever adds a node to a cluster whose join information is already available.

1.4 When to use which

You want to...	Use this, applied here
Stand up a brand-new cluster from bare hosts	<code>RemoteCluster</code> , <code>nodeInfo.nodeType: control-plane</code> , on the management cluster
Add a worker while you're still building out a new cluster	<code>RemoteCluster</code> , <code>nodeInfo.nodeType: worker</code> , on the management cluster
Add on-prem/bare-metal capacity to a cluster that's already running	<code>NodeProvision</code> , <code>provider: OnPrem</code> , on the remote cluster
Burst into the cloud with extra capacity	<code>NodeProvision</code> , <code>provider: AWS</code> , on the remote cluster
Rotate registry credentials, change the VPN range, or bump the Kubernetes version for future nodes	Edit <code>NodeProvisionNetConfig</code> (or the parent <code>RemoteCluster</code> , which keeps it in sync)

2. Architecture & Diagrams

2.1 Overall architecture

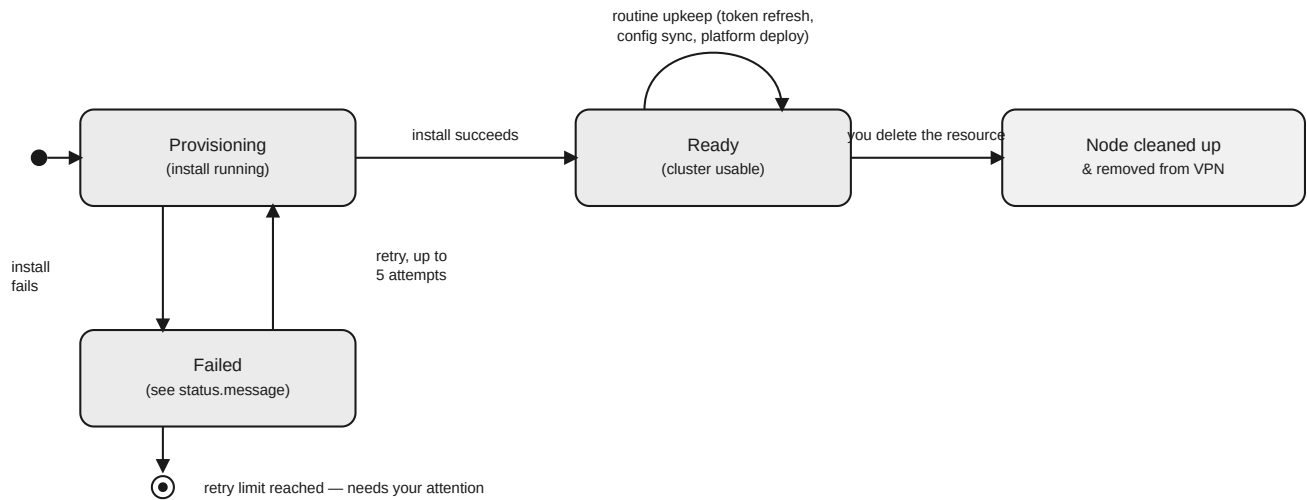


Numbered flow:

1. Cluster bootstrap connects over SSH to install Kubernetes, the container runtime, networking (over the WireGuard VPN), and — for control planes — a GitOps agent plus platform baseline.
2. On success, it writes the cluster's shared configuration (join command, VPN range, software versions, and — if configured — registry credentials) to that cluster's own `NodeProvisionNetConfig`.
3. Cluster scale-out, running on the remote cluster, reads that shared configuration whenever it needs a join command, VPN server details, or registry credentials for a **new** node.
4. Back on the management cluster, cluster bootstrap deploys your GitOps platform stack onto the new cluster (only if you opted into that).
5. Cluster scale-out provisions the new node — over SSH for on-prem hosts, or by launching an EC2 instance for AWS — and joins it using the cached join command.
6. Once joined, cluster scale-out refreshes the cluster's own join token directly against its own Kubernetes API (no dependency on the management cluster) and labels the new node so GPU/CPU- specific workloads schedule onto it correctly.

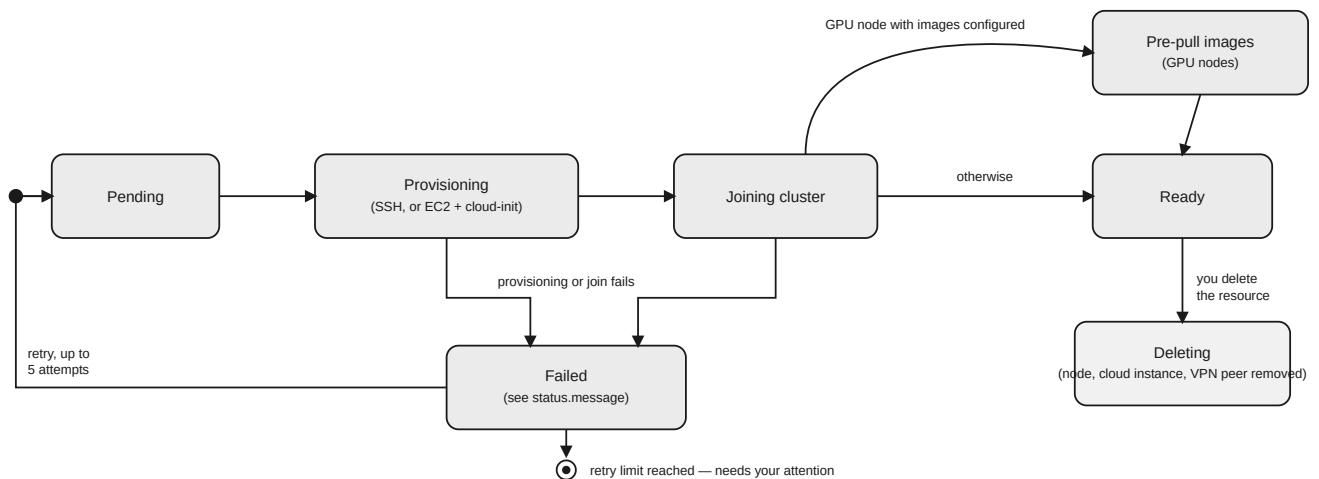
Both controllers register and remove WireGuard peers on the VPN server automatically as nodes are added and removed.

2.2 RemoteCluster : what happens, phase by phase



RemoteCluster moves Provisioning → Ready, retries up to 5 times on failure, and is cleaned up on deletion.

2.3 NodeProvision : what happens, phase by phase



NodeProvision validates, provisions via SSH or EC2, joins the cluster, optionally pre-pulls GPU images, then becomes Ready.

3. Prerequisites

3.1 Cluster software and versions

- A running management-cluster **Kubernetes API** to install the CRDs and the controller into.
- **Target Kubernetes version for the clusters you're building** — set per cluster via `kubernetesVersion` (e.g. `v1.34.2`) in your `RemoteCluster` / `NodeProvisionNetConfig` — this is entirely independent of whatever Kubernetes version your management cluster runs.
- **Nephio/Porch**, on the management cluster, **only if** you turn on `gitConfig.enable: "true"` to have the platform stack deployed via GitOps.

3.2 Environment/configuration

- **SSH reachability** — the controller must be able to reach every host you list (by LAN IP or its VPN IP) on the SSH port you configure (default `22`). There's no bastion/jump-host support; it connects directly.
- **Passwordless sudo** for the SSH user on every host you provision.
- **Ubuntu 22.04 (Jammy)** on every host that becomes a cluster node.
- A **WireGuard VPN server** reachable over SSH — see [WIREGUARD_SETUP.md](#) if you need to stand one up.
- **GPU nodes**: NVIDIA drivers must already be installed on the host beforehand — this project does not install GPU drivers for you.
- **AWS credentials**, if you'll provision nodes with `provider: AWS` — either an access key/secret in a Secret, or an IAM role attached to the controller's pod.

3.3 Permissions

The controller needs a `ClusterRole` covering the `RemoteCluster`, `NodeProvision`, and `NodeProvisionNetConfig` resources (including their status), Kubernetes `Node` objects, `Secret` s and `ConfigMap` s (including in `kube-system / kube-public`, for join-token management), `Job` s (for GPU image pre-pulling), and — only if you enable GitOps deployment — the Nephio/Porch resource types. All of this is already defined in the manifests you'll apply in §4; you don't need to hand-write any RBAC.

Security note: SSH connections from the controller do not verify host keys. Make sure the network path to your nodes and VPN server (e.g. the VPN tunnel itself) is one you trust.

4. Installation

This guide assumes the controller container image is already built and pushed somewhere your cluster can pull it from. If you don't have that yet, ask whoever manages your image builds for the image reference before continuing.

4.1 Install the CRDs

```
make install
```

This applies the three CRDs to your management cluster:

- RemoteCluster
- NodeProvision
- NodeProvisionNetConfig

Verify:

```
kubectl get crds | grep -E 'dcn.ssu.ac.kr'
```

Expected output:

```
nodeprovisionnetconfigs.ml.dcn.ssu.ac.kr    2026-01-01T00:00:00Z
nodeprovisions.ml.dcn.ssu.ac.kr             2026-01-01T00:00:00Z
remoteclusters.infra.dcn.ssu.ac.kr          2026-01-01T00:00:00Z
```

4.2 Deploy the controller

```
make deploy IMG=<registry>/remote-cluster-provisioner:<tag>
```

Point `IMG` at the pre-built image you were given. This creates:

- Namespace `remote-cluster-provisioner-system`
- The controller's service account and permissions
- Deployment/`remote-cluster-provisioner-controller-manager` (1 replica)

The same steps (§4.1 and §4.2) are what you'll also run **on every remote cluster** you create, before you start applying `NodeProvision` resources there.

4.3 Apply your first resources

```
# On the management cluster – SSH credentials + a RemoteCluster
kubectl apply -f config/samples/infra_v1_remoteclasser_gpu_worker.yaml
```

```
# On the remote cluster, once it exists – add a node via NodeProvision
kubectl apply -f config/samples/ml_v1alpha1_nodeprovision.yaml
```

(See §6 for a full walk-through with expected output at each step.)

4.4 Verify the installation

```
# Controller pod is Running
kubectl get pods -n remote-cluster-provisioner-system

# Health/readiness endpoints (port-forward first)
kubectl port-forward -n remote-cluster-provisioner-system \
  deployment/remote-cluster-provisioner-controller-manager 8083:8083
curl -sf localhost:8083/healthz && echo OK
curl -sf localhost:8083/readyz && echo OK

# CRDs are established
kubectl get crd remoteclusters.infra.dcn.ssu.ac.kr \
  -o jsonpath='{.status.conditions[?(@.type=="Established")].status}'
```

5. Usage

5.1 RemoteCluster — create/manage a cluster from the management cluster

Full field reference is in §10.1. Minimal control-plane example:

```
apiVersion: infra.dcn.ssu.ac.kr/v1
kind: RemoteCluster
metadata:
  name: ml-cluster-cp
spec:
  clusterName: ml-cluster      # shared across every node in this logical
cluster
  host: 192.168.3.234
  port: "22"
  user: ubuntu
  nodeInfo:
    nodeType: control-plane    # or: worker
    hardwareType: cpu          # or: gpu
```



```

softwareConfig:
  kubernetesVersion: v1.34.2
  cnlabRuntime:
    registry: ghcr.io
    repository: vitu-mafeni/cnlab-runtime
    version: 1.0.0-beta
    orasVersion: 1.3.2
    credentialsRef:
      name: cnlab-runtime-registry
      namespace: default
auth:
  sshPrivateKeySecretRef:
    name: cp-node-ssh-secret
    key: id_rsa
vpnConfig:
  ip: 10.9.0.13
  vpnServerPublicIP: 13.215.206.108
  vpnServerSSHPort: "22"
  vpnServerSSHUsername: ubuntu
  vpnSshCredentialsRef:
    name: vpn-server-ssh-secret
    namespace: default
    key: id_rsa
gitConfig:
  enable: "true" # omit/false to skip GitOps platform deployment
entirely
  gitServer: "http://192.168.3.99:31810"
  gitUsername: "nephio"
  upstreamPlatformRepo: "catalog-workloads-mlplatform"
  packageRevision: "v1.0.0"

```

Key fields to get right:

- **nodeInfo.nodeType** — `control-plane` installs a brand-new cluster on this host; `worker` joins this host to whichever `RemoteCluster` in the same namespace has a matching `spec.clusterName` and `nodeType: control-plane`. If that control-plane isn't `Ready` yet, the worker just waits and checks again periodically.
- **auth** — exactly one of `sshPrivateKeySecretRef` or `passwordSecretRef`. Key auto-detection: a secret value starting with `-----BEGIN` is treated as a private key; anything else as a password.
- **vpnConfig.ip** — once set, the controller talks to the node over its VPN IP instead of `spec.host`, which keeps working even after the node moves onto a different network path.
- **gitConfig.enable** — a `string` `"true"` / `"false"`, not a bare `true` / `false`. Only when `"true"` does the platform stack get deployed via GitOps.

5.2 NodeProvisionNetConfig — shared cluster settings

Normally created and kept up to date for you automatically (see §8.1), but you can also apply it directly on a remote cluster you're testing in isolation:

```
apiVersion: ml.dcn.ssu.ac.kr/v1alpha1
kind: NodeProvisionNetConfig
metadata:
  name: my-cluster-netconfig
spec:
  clusterName: my-cluster
  vpnRange: "10.9.0.0/24"
  vpnServerPublicConfig:
    publicIP: "13.215.206.108"
    sshPort: 22
    sshUsername: ubuntu
    vpnPort: 51820
    vpnSshCredentialsRef:
      name: vpn-server-secret
      namespace: default
      key: id_rsa
  softwareConfig:
    kubernetesVersion: "v1.34.2"
    cnlabRuntime:
      registry: "ghcr.io"
      repository: "vitu-mafeni/cnlab-runtime"
      version: "1.0.0-beta"
      orasVersion: "1.3.2"
      credentialsRef:
        name: cnlab-runtime-registry
        namespace: default
```

Some sample files also show `softwareConfig.nvidiaDriverVersion`, `nvidiaContainerToolkitVersion`, and `k8sDevicePluginVersion`. **These aren't supported fields** today — don't rely on them. GPU driver/toolkit versions aren't configurable through this resource; install them on the host yourself beforehand.

5.3 NodeProvision — add a node from the remote cluster

On-prem (an existing host, reached over SSH):

```
apiVersion: ml.dcn.ssu.ac.kr/v1alpha1
kind: NodeProvision
```

```

metadata:
  name: gpu-worker-01
  namespace: default
spec:
  provider: OnPrem
  role: worker
  hardwareType: gpu          # controls which pre-pull images target this
node
  nodeLabel: gpu
  ipAddress: 192.168.28.150
  sshPort: 22
  sshUsernameOverride: ubuntu
  credentialsRef:
    name: gpu-worker-01-ssh-secret
    namespace: default
    key: id_rsa

```

AWS (a new EC2 instance — most fields auto-resolve):

```

apiVersion: ml.dcn.ssu.ac.kr/v1alpha1
kind: NodeProvision
metadata:
  name: aws-node-001
  namespace: default
spec:
  provider: AWS
  role: worker
  nodeLabel: cpu             # "cpu" → t3.xlarge | "gpu" → p3.2xlarge (auto-
resolved)
  region: ap-northeast-2
  credentialsRef:
    name: aws-node-credentials
    namespace: default
  # awsConfig is entirely optional – every field below is auto-populated when
omitted:
  # awsConfig:
  #   instanceType: p3.2xlarge  # overrides the nodeLabel → instance-type lookup
  #   ami: ami-0c9c942bd7bf113a2
  #   vpcId: vpc-xxxxxxx
  #   subnetId: subnet-xxxxxxx
  #   securityGroupIds: [sg-xxxxxxx]
  #   keyPairName: my-keypair
  #   iamInstanceProfile: my-profile

```

```
# rootVolumeSizeGB: 100
# tags: {environment: production}
```

Important fields:

- **nodeLabel** drives the AWS defaults — "cpu" → t3.xlarge , "gpu" → p3.2xlarge . Set `spec.awsConfig.instanceType` (or the top-level `spec.instanceType`) to pick an exact instance type instead.
- **hardwareType** (separate from `nodeLabel`) controls which pre-pull images target this node — only relevant for `gpu` nodes when your cluster config lists images to pre-pull.
- **role** — always `worker` in practice; this resource only ever joins nodes to an existing cluster, it never bootstraps a brand-new one (that's what `RemoteCluster` is for).
- **credentialsRef.key** — if omitted, the controller tries a few common key names automatically (`privateKey` , `id_rsa` , `ssh-privatekey` , `password` , `key`).

6. Examples

6.1 End-to-end: bootstrap a control plane, add a worker, scale with `NodeProvision`

Step 1 — apply the SSH secret and control-plane `RemoteCluster` (on the management cluster):

```
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: cp-node-ssh-secret
type: Opaque
stringData:
  id_rsa: |
    -----BEGIN OPENSSH PRIVATE KEY-----
    ...
    -----END OPENSSH PRIVATE KEY-----
---
apiVersion: infra.dcn.ssu.ac.kr/v1
kind: RemoteCluster
metadata:
  name: ml-cluster-cp
spec:
  clusterName: ml-cluster
  host: 192.168.3.234
  port: "22"
  user: ubuntu
  nodeInfo:
    nodeType: control-plane
    hardwareType: cpu
```

```
softwareConfig:
  kubernetesVersion: v1.34.2
auth:
  sshPrivateKeySecretRef:
    name: cp-node-ssh-secret
    key: id_rsa
EOF
```

Expect: `status.phase` moves `""` → `Provisioning` immediately, then stays `Provisioning` for roughly 5–15 minutes while the control plane is installed. Watch it:

```
kubectl get remotecluster ml-cluster-cp -w
```

NAME	PHASE	MESSAGE
ml-cluster-cp	Provisioning	Provisioning in progress
ml-cluster-cp	Ready	Provisioned

Controller logs during this step (`kubectl logs -n remote-cluster-provisioner-system deployment/remote-cluster-provisioner-controller-manager -f`) — an **illustrative transcript** of what you'll typically see, for a cluster with `vpnConfig` configured (as in the full samples under `config/samples/`). Exact timestamps and the number of "in progress" lines will vary with how long the install actually takes on your hardware:

```
2026-09-08T10:15:03.412+0900 INFO    remoteclasser Starting provisioning node
for cluster {"cluster": "ml-cluster-cp", "clusterName": "ml-cluster",
"nodeType": "control-plane", "phase": ""}
2026-09-08T10:15:03.498+0900 INFO    remoteclasser Control plane init
goroutine started {"cluster": "ml-cluster-cp", "clusterName": "ml-cluster",
"startPhase": 0}
2026-09-08T10:15:33.501+0900 INFO    remoteclasser Control plane init in
progress, requeueing {"cluster": "ml-cluster-cp", "clusterName": "ml-
cluster"}
2026-09-08T10:16:03.512+0900 INFO    remoteclasser Control plane init in
progress, requeueing {"cluster": "ml-cluster-cp", "clusterName": "ml-
cluster"}
                                : (repeats roughly every 30s while
Kubernetes, the container runtime, networking, and ArgoCD are installed over SSH;
typically 5-15 minutes) :
2026-09-08T10:23:41.220+0900 INFO    remoteclasser Control plane init
completed {"cluster": "ml-cluster-cp", "clusterName": "ml-cluster",
"joinCommand": true}
```

```

2026-09-08T10:23:42.005+0900 INFO    remoteclasser  Kubeadm bootstrap token
has never been explicitly refreshed; will refresh now {"cluster": "ml-cluster-
cp", "clusterName": "ml-cluster"}
2026-09-08T10:23:44.115+0900 INFO    remoteclasser  Refreshed kubeadm
bootstrap token      {"cluster": "ml-cluster-cp", "clusterName": "ml-cluster"}
2026-09-08T10:23:44.900+0900 INFO    remoteclasser  Creating PackageVariants
{"cluster": "ml-cluster-cp", "clusterName": "ml-cluster"}
2026-09-08T10:23:45.760+0900 INFO    remoteclasser  Core PackageVariants
created – requeueing to allow platform sync before overlay step {"cluster":
"ml-cluster-cp", "clusterName": "ml-cluster", "delay": "2m0s"}
2026-09-08T10:25:46.003+0900 INFO    remoteclasser  Creating PackageVariants
{"cluster": "ml-cluster-cp", "clusterName": "ml-cluster"}
2026-09-08T10:25:47.410+0900 INFO    remoteclasser  PackageVariants created;
cluster is fully ready      {"cluster": "ml-cluster-cp", "clusterName": "ml-
cluster"}
2026-09-08T10:25:48.020+0900 INFO    remoteclasser  Kubeadm bootstrap token
still valid      {"cluster": "ml-cluster-cp", "clusterName": "ml-cluster",
"refreshedAt": "2026-09-08T10:23:44+09:00", "nextRefreshIn": "22h58m0s"}
2026-09-08T10:25:48.021+0900 INFO    remoteclasser  Cluster fully ready
{"cluster": "ml-cluster-cp", "clusterName": "ml-cluster", "nextTokenRefreshIn":
"22h58m0s"}

```

A couple of lines appear more than once above — that's expected, not a duplicate or a retry: the platform stack is deployed in two waves (core, then the rest), and each wave's completion triggers its own follow-up check.

Step 2 — add a worker to the same `clusterName` :

```

kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: worker-01-ssh-secret
type: Opaque
stringData:
  id_rsa: |
    -----BEGIN OPENSSH PRIVATE KEY-----
    ...
    -----END OPENSSH PRIVATE KEY-----
---
apiVersion: infra.dcn.ssu.ac.kr/v1
kind: RemoteCluster
metadata:
  name: ml-cluster-worker-01

```

```
spec:
  clusterName: ml-cluster          # must match the control-plane's clusterName
  host: 192.168.3.235
  port: "22"
  user: ubuntu
  nodeInfo:
    nodeType: worker
    hardwareType: gpu
  auth:
    sshPrivateKeySecretRef:
      name: worker-01-ssh-secret
      key: id_rsa
EOF
```

Expect: once the control plane is confirmed `Ready`, this host is joined to it. On success the resulting Kubernetes node is labeled so GPU-specific workloads (like image pre-pull jobs) can schedule onto it.

Controller logs during this step — illustrative transcript, same caveats as Step 1. Note the gap: joining a worker happens as one continuous step, so there's no repeated "in progress" line — the log simply goes quiet for a few minutes between the first line and the last:

```
2026-09-08T10:31:02.104+0900 INFO    remoteccluster  Synced VPN server config
from control-plane      {"cluster": "ml-cluster-worker-01", "clusterName": "ml-
cluster", "cp": "ml-cluster-cp"}

                                : (no further log lines while the join
runs over SSH — typically a few minutes) :
2026-09-08T10:34:18.760+0900 INFO    remoteccluster  Worker node joined to
cluster {"cluster": "ml-cluster-worker-01", "clusterName": "ml-cluster"}
2026-09-08T10:34:19.902+0900 INFO    remoteccluster  Labeled worker node for
DaemonSet targeting     {"cluster": "ml-cluster-worker-01", "node": "ml-cluster-
worker-01", "hardwareType": "gpu"}
```

Step 3 — scale out with `NodeProvision`, applied **on the remote cluster** (`ml-cluster`) once it's up:

```
kubectl --kubeconfig=ml-cluster.kubeconfig apply -f
config/samples/ml_v1alpha1_nodeprovision.yaml
```

That sample file applies, in order: the `NodeProvision` resource (`provider: OnPrem`), its SSH Secret, a `NodeProvisionNetConfig` (harmless to re-apply — it will already exist from Step 1/2), the registry credentials Secret, and the VPN server SSH Secret.

```
kubectl --kubeconfig=ml-cluster.kubeconfig get nodeprovision gpu-worker-01 -w
```

NAME	PHASE	MESSAGE
gpu-worker-01	Pending	
gpu-worker-01	Provisioning	Validation successful
gpu-worker-01	Bootstrapping	On-prem bootstrap goroutine started
gpu-worker-01	RegisteringNode	
gpu-worker-01	Joining	
gpu-worker-01	Ready	Node reached Ready state

Controller logs during this step — illustrative transcript, same caveats as Step 1, this time on the **remote cluster's** controller:

```
2026-09-08T11:02:10.301+0900 INFO    nodeprovision  Request received, starting
provisioning      {"nodeprovision": "gpu-worker-01", "provider": "OnPrem"}
2026-09-08T11:02:10.940+0900 INFO    nodeprovision  Validation successful
{"nodeprovision": "gpu-worker-01"}
2026-09-08T11:02:11.205+0900 INFO    nodeprovision  On-prem bootstrap
goroutine started {"nodeprovision": "gpu-worker-01"}
                        : (installing Kubernetes and the
container runtime over SSH – typically several minutes) :
2026-09-08T11:07:52.114+0900 INFO    nodeprovision  On-prem bootstrap
completed        {"nodeprovision": "gpu-worker-01"}
2026-09-08T11:07:52.560+0900 INFO    nodeprovision  On-prem node provisioned,
waiting for cluster join {"nodeprovision": "gpu-worker-01"}
2026-09-08T11:08:22.601+0900 INFO    nodeprovision  Node not yet visible in
cluster – waiting for kubelet to register {"nodeprovision": "gpu-worker-01"}
2026-09-08T11:08:52.744+0900 INFO    nodeprovision  Node registered with
control plane    {"nodeprovision": "gpu-worker-01", "nodeName": "gpu-worker-
01"}
2026-09-08T11:08:53.310+0900 INFO    nodeprovision  Stamped ownership metadata
on node         {"nodeprovision": "gpu-worker-01", "hardwareType": "gpu"}
2026-09-08T11:08:53.815+0900 INFO    nodeprovision  Node reached Ready state
{"nodeprovision": "gpu-worker-01"}
```

Confirm the node joined:

```
kubectl --kubeconfig=ml-cluster.kubeconfig get nodes -l
```



```
infra.dcn.ssu.ac.kr/worker=true
```

6.2 Example: rotating registry credentials

```
kubectl apply -f - <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: cnlab-runtime-registry
type: Opaque
stringData:
  username: "your-github-username"
  token: "ghp_newtoken..."
EOF
```

No restart needed. The change is picked up automatically — the management cluster pushes it out to the remote cluster's shared config, and each node re-authenticates with the registry the next time it checks.

Expected log line (management side):

```
INFO cnlab-runtime credentials changed – syncing to remote cluster
```

Expected log line (remote side, per node):

```
INFO Runtime registry credentials changed – syncing to node via oras login
INFO Runtime registry credentials synced to node
```

7. Logs & Troubleshooting

7.1 Viewing/filtering controller logs

```
# Follow the controller (same command on both the management cluster
# and every remote cluster – namespace is always
# remote-cluster-provisioner-system)
kubectl logs -n remote-cluster-provisioner-system \
```

```

deployment/remote-cluster-provisioner-controller-manager -f

# Filter for a specific RemoteCluster or NodeProvision by name
kubectl logs -n remote-cluster-provisioner-system \
  deployment/remote-cluster-provisioner-controller-manager | grep 'ml-cluster-cp'

# Filter for failures only
kubectl logs -n remote-cluster-provisioner-system \
  deployment/remote-cluster-provisioner-controller-manager | grep -i error

```

By default, logs print in a human-readable console format (timestamp, level, a short logger name, the message, then any extra details as key/value pairs) rather than JSON — that's fine to read directly, or to pipe through `grep` / `jq` -style tools if your log format needs adjusting for your log pipeline.

7.2 Representative example logs

Labeled **representative** — these are typical messages you'll see, but the exact surrounding details (cluster name, attempt numbers, timestamps) will vary per run.

Normal cluster provisioning:

```

INFO Starting provisioning node for cluster           cluster=ml-cluster-cp
clusterName=ml-cluster nodeType=control-plane
INFO Control plane init goroutine started           startPhase=0
INFO Control plane init in progress, requeueing
INFO Control plane init completed                   joinCommand=true
INFO Cluster fully ready
nextTokenRefreshIn=23h0m0s

```

Join-token refresh (automatic, roughly every 23 hours):

```

INFO Kubeadm bootstrap token due for refresh
INFO Refreshed kubeadm bootstrap token
INFO Kubeadm bootstrap token still valid             refreshedAt=...
nextRefreshIn=21h0m0s

```

Retry / terminal failure:

```
ERROR RemoteCluster provisioning failed – will retry      attempt=2 maxRetries=5
ERROR RemoteCluster provisioning reached retry limit – no further retries
attempts=5 maxRetries=5
```

Registry credential sync:

```
INFO cnlab-runtime credentials changed – syncing to remote cluster
ERROR cnlab-runtime credential sync reached retry limit – no further retries
```

Normal on-prem node provisioning:

```
INFO Request received, starting provisioning
INFO Validation successful
INFO On-prem bootstrap goroutine started
INFO On-prem bootstrap completed
INFO On-prem node provisioned, waiting for cluster join
INFO Node not yet visible in cluster – waiting for kubelet to register
INFO Node registered with control plane
INFO Stamped ownership metadata on node
INFO Node reached Ready state
```

AWS node provisioning:

```
INFO AWS validation successful
INFO Resolved instance type from nodeLabel
INFO Resolved AMI
INFO Resolved network config
INFO Resolved EC2 key pair
INFO Creating EC2 instance
INFO EC2 instance created
INFO EC2 instance created, waiting for it to become running
INFO EC2 instance running
INFO Executing cloud-init bootstrap
```

Stall / retry:

```
INFO NodeProvision stalled without an InstanceID, failing for retry
INFO On-prem bootstrap stalled with no progress, failing for retry
INFO Node registration stalled, failing for retry
INFO NodeProvision in terminal Failed state – retry limit reached, manual
intervention required
INFO Retrying NodeProvision after failure – releasing stale VPN IP and resetting
phase
```

Deletion:

```
INFO Deprovisioning RemoteCluster
INFO Resetting node via SSH
INFO Node reset complete
INFO Removed WireGuard peer from VPN server
INFO RemoteCluster cleanup complete
```

7.3 Common errors & how to resolve them

Symptom / log message	Likely cause	Resolution
<code>SSHConnectionFailed</code> in <code>status.message</code>	Host unreachable, wrong credentials, or the VPN isn't up yet	Verify <code>spec.host</code> / <code>spec.vpnConfig.ip</code> is reachable; check the Secret's key name matches what you referenced
<code>RemoteCluster</code> provisioning reached retry limit – no further retries	5 consecutive failures	Fix the underlying issue (see <code>status.message</code>), then reset: <code>kubectl patch remotecluster <name> --subresource=status --type=merge -p '{"status":{"provisionRetryCount":0}}'</code>
<code>NodeProvision</code> in terminal <code>Failed</code> state	5 consecutive failures on the remote side	<code>kubectl patch nodeprovision <name> --subresource=status --type=merge -p '{"status":{"provisionRetryCount":0}}'</code>
<code>cnlab-runtime</code> credential sync reached retry limit	VPN link between clusters may be down, or the credentials Secret is malformed	Check VPN connectivity; reset via <code>kubectl patch remotecluster <name> --subresource=status --type=merge -p '{"status":{"cnlabSyncRetryCount":0}}'</code>
Node stuck in <code>Bootstrapping</code>	Install is legitimately still running (5–15 min is normal), or it's genuinely hung	<code>kubectl get nodeprovision <name> -o jsonpath='{.status.message}'</code> ; SSH in and check <code>sudo fuser /var/lib/dpkg/lock-frontent</code> and <code>tail -50 /var/log/node-provision.log</code>
Registry pull returns <code>unauthorized</code>	Registry credentials missing/empty in the cluster's shared config	Apply the registry credentials Secret (or re-apply the sample, which includes it)
Networking plugins missing after cluster install	An install step was interrupted	Manually install the CNI plugins bundle (<code>v1.5.1</code>) to <code>/opt/cni/bin</code> on the affected node
GitOps platform resources not appearing / stuck	The platform hasn't synced the new cluster's repository yet, or a stale	Check your Nephio/Porch resources; delete stale ones to force re-creation

Symptom / log message	Likely cause	Resolution
	resource exists from a previous attempt	
Login page works after cluster is Ready, but auth fails	The identity provider's Service selector doesn't match its pod labels	Patch the Service's selector to match (see your platform's docs)
A <code>NodeProvision</code> never advances past <code>RegisteringNode</code>	Node isn't visible yet — kubelet failed to register, or its VPN IP doesn't match any Node's address	SSH into the node, check the Kubernetes agent status, and check the VPN peer list on both the node and VPN server
<code>not yet implemented</code> error for a GCP/Azure provider	Only <code>AWS</code> and <code>OnPrem</code> are supported today	Use <code>AWS</code> or <code>OnPrem</code>

7.4 Debugging / diagnostic commands

```
# Full status block for a RemoteCluster or NodeProvision
kubectl get remotecluster <name> -o yaml
kubectl get nodeprovision <name> -o yaml

# Full condition history (this list keeps every entry, not just the latest)
kubectl get remotecluster <name> -o jsonpath='{.status.conditions}' | jq

# Check what a stuck on-prem NodeProvision is doing
kubectl get nodeprovision <name> -o jsonpath='{.status.progress}{"\n"}{.status.message}'

# Inspect the image pre-pull job (GPU nodes only)
kubectl get jobs -l job-name=<nodeprovision-name>-prepull
kubectl logs job/<nodeprovision-name>-prepull

# On the remote control-plane host directly (bypasses the controller entirely)
ssh ubuntu@<cp-ip> 'sudo tail -50 /var/log/node-provision.log'
ssh ubuntu@<cp-ip> 'sudo crictl info'
ssh ubuntu@<cp-ip> 'wg show wg0'
```

8. Controller Lifecycle / Reconciliation

8.1 RemoteCluster create/update/delete

Create: the resource moves to `Provisioning`. For a **control-plane** node, Kubernetes, container runtime, networking, and (if enabled) the GitOps agent are installed — this typically takes 5–15 minutes and can be watched via `status.phase / status.message`. For a **worker**, it's joined to the sibling `RemoteCluster` with the same `clusterName` — this requires that control-plane to already be `Ready`. On success, the cluster's shared configuration (`NodeProvisionNetConfig`) is created/updated, and the resource moves to `Ready`. For a control-plane, this is followed by deploying the GitOps platform stack in two waves.

Update: a `Ready` cluster is periodically re-checked to keep its join token fresh (roughly every 23 hours) and to catch any configuration drift (VPN settings, software versions, registry credentials) between the `RemoteCluster` resource and the cluster's actual shared config. If a provisioning attempt is interrupted partway (e.g. a controller restart), the next attempt resumes from where it left off rather than starting the entire install over.

Delete: deleting a `RemoteCluster` cleanly tears the node back down — it's drained and removed from the cluster (workers only), reset back to a bare host (Kubernetes, container runtime, and networking uninstalled), and removed from the VPN. All of this cleanup is best-effort: an unreachable or already-gone node never blocks the resource from being deleted.

8.2 NodeProvision create/update/delete

Create: the resource works through the phases shown in §2.3. For AWS, unset fields (instance type, AMI, networking, key pair) are automatically resolved for you. Once the node is visible in the cluster, it's labeled so GPU/CPU-specific workloads schedule onto it correctly.

Update: a `Ready` node is periodically checked for registry-credential drift and re-authenticates automatically if credentials changed. A `Failed` node below the retry limit is automatically reset for a clean retry — you don't need to delete and recreate it.

Delete: the node is removed from the cluster, its cloud instance is terminated (AWS) or it's reset back to a bare host (on-prem), and it's removed from the VPN.

8.3 Retries, failures, and recovery

Every provisioning failure is counted. After **5 consecutive failures**, the resource is left in a terminal `Failed` state and stops retrying automatically — this is intentional, since repeated failures against the same host usually mean something needs a human to look at (wrong credentials, host down, disk full, etc.). Once you've fixed the underlying issue, reset the retry counter to resume — the exact `kubectl patch` command for each resource type is in §7.3.

`RemoteCluster.status.conditions` keeps a full history, not just the latest state — useful for seeing exactly what happened across the resource's lifetime, not only its current status.

9. Operations

9.1 Health checks

```
kubectl get pods -n remote-cluster-provisioner-system
kubectl describe pod -n remote-cluster-provisioner-system \
  -l control-plane=controller-manager
```

The liveness/readiness probes only confirm the controller process itself is alive — not that any particular SSH/AWS/VPN operation is succeeding. For that, check the resource's own `status / conditions` and logs (see [§7](#)).

9.2 Restart

```
kubectl rollout restart deployment/remote-cluster-provisioner-controller-manager \
  -n remote-cluster-provisioner-system
```

In-flight provisioning work resumes from where it left off after a restart rather than starting over — see [§8.1/§8.2](#).

9.3 Upgrade

```
make deploy IMG=<registry>/remote-cluster-provisioner:<new-tag>
```

Point `IMG` at whichever new pre-built image you've been given. If the new version adds new CRD fields, re-run `make install` first so the CRDs are up to date before the new controller version starts.

9.4 Uninstall

```
# Remove the controller and its permissions/namespace
make undeploy

# Remove the CRDs (only after – see the warning below)
make uninstall
```


Before removing anything, delete any live `RemoteCluster` / `NodeProvision` resources first (`kubectl delete remotecluster --all` , `kubectl delete nodeprovision --all`) and wait for them to finish deleting, so nodes get reset and removed from the VPN properly. Removing the CRDs while resources still exist skips that cleanup — you'd be left with nodes still joined to the cluster and still registered on the VPN.

9.5 Inspecting status across your fleet

```
# All RemoteClusters and their current phase
kubectl get remotecluster -o custom-
columns=NAME:.metadata.name,PHASE:.status.phase,CLUSTER:.spec.clusterName

# All NodeProvisions on a remote cluster
kubectl get nodeprovision -o custom-
columns=NAME:.metadata.name,PHASE:.status.phase,PROVIDER:.spec.provider,IP:.status.ipAddress
```

9.6 Monitoring

Day-to-day observability is status/condition- and log-driven (see [§7](#) and [§9.5](#)) rather than dashboard-driven — there isn't a bundled dashboard today. If your platform team wants Prometheus metrics, ask them to enable the metrics endpoint on the controller Deployment.

10. Reference

10.1 RemoteCluster

Spec:

Field	Type	Notes
<code>clusterName</code>	string	Shared across every node (control-plane + workers) of one logical cluster
<code>host</code>	string	LAN/public IP or hostname of the target node
<code>port</code>	string	SSH port (e.g. "22")
<code>user</code>	string	SSH username
<code>nodeInfo.nodeType</code>	string	<code>control-plane</code> <code>worker</code>
<code>nodeInfo.hardwareType</code>	string	<code>cpu</code> <code>gpu</code>
<code>nodeInfo.softwareConfig.kubernetesVersion</code>	string	e.g. <code>v1.34.2</code>
<code>nodeInfo.softwareConfig.imagePrepulls[]</code>	<code>{image, nodeTarget}</code>	<code>nodeTarget</code> is <code>gpu</code> or <code>all</code> (default <code>all</code>)
<code>nodeInfo.softwareConfig.imagePullSecretRef</code>	secret ref	optional
<code>nodeInfo.softwareConfig.cnlabRuntime</code>	object	optional; sensible defaults if omitted
<code>nodeInfo.softwareConfig.platformVariables[]</code>	<code>{key, value}</code>	optional; passed through to the GitOps platform stack
<code>auth.sshPrivateKeySecretRef</code> / <code>auth.passwordSecretRef</code>	secret ref	exactly one; auto-detects key vs. password auth
<code>vpnConfig.ip</code>	string	this node's VPN IP; preferred over <code>host</code> once set
<code>vpnConfig.vpnServerPublicIP</code> / <code>vpnServerSSHPort</code> / <code>vpnServerSSHUsername</code>	string	VPN server SSH connection details

Field	Type	Notes
<code>vpnConfig.vpnSshCredentialsRef</code>	secret ref	Secret with the VPN server's SSH credential
<code>gitConfig.enable</code>	string	"true" / "false" — gates GitOps platform deployment
<code>gitConfig.gitServer</code> / <code>gitUsername</code> / <code>upstreamPlatformRepo</code> / <code>packageRevision</code>	string	GitOps source details

Status:

Field	Meaning
<code>phase</code>	Provisioning → Ready → Failed
<code>message</code>	Human-readable status, includes retry count on failure
<code>conditions[]</code>	Full history of status changes, most recent last
<code>joinCommand</code>	Cached join command, used when adding workers
<code>provisionRetryCount</code>	Consecutive provisioning failures; resets to 0 on Ready
<code>cnlabSyncRetryCount</code>	Consecutive registry-credential sync failures

10.2 NodeProvision

Spec:

Field	Type	Notes
<code>provider</code>	string	<code>OnPrem</code> <code>AWS</code> (<code>GCP</code> / <code>Azure</code> are not implemented yet)
<code>role</code>	string	always <code>worker</code> in practice
<code>hardwareType</code>	string	<code>gpu</code> / <code>cpu</code> (or empty) — filters which images get pre-pulled
<code>nodeLabel</code>	string	drives AWS default instance-type resolution (<code>cpu</code> → <code>t3.xlarge</code> , <code>gpu</code> → <code>p3.2xlarge</code>)
<code>region</code>	string	AWS region
<code>instanceType</code>	string	overrides the <code>nodeLabel</code> lookup
<code>hostname</code> / <code>ipAddress</code>	string	on-prem target; <code>ipAddress</code> takes priority if both set
<code>sshPort</code>	int	default <code>22</code>
<code>sshUsernameOverride</code>	string	
<code>credentialsRef</code>	secret ref	key auto-detected if omitted
<code>awsConfig</code>	object	<code>vpcId</code> , <code>subnetId</code> , <code>securityGroupIds[]</code> , <code>ami</code> , <code>keyPairName</code> , <code>iamInstanceProfile</code> , <code>tags{}</code> , <code>rootVolumeSizeGB</code> — all auto-resolved if omitted

Status:

Field	Meaning
<code>phase</code>	See §2.3
<code>message</code>	Human-readable status
<code>instanceId</code> / <code>hostname</code> / <code>ipAddress</code>	Identity
<code>publicIp</code> / <code>privateIp</code>	AWS-only
<code>vpnIp</code>	VPN IP allocated for this node
<code>progress</code>	0–100
<code>nodeName</code>	Kubernetes node name once registered
<code>runtimeCredentialsHash</code>	Fingerprint of last-synced registry credentials
<code>provisionRetryCount</code>	Consecutive failures; resets to 0 on <code>Ready</code>

10.3 `NodeProvisionNetConfig`

Spec:

Field	Type	Notes
<code>clusterName</code>	string	
<code>vpnRange</code>	string	CIDR, e.g. <code>10.9.0.0/24</code>
<code>vpnServerPublicConfig.publicIP</code>	string	
<code>vpnServerPublicConfig.sshPort</code> / <code>sshUsername</code>	string	defaults <code>22</code> / <code>ubuntu</code>
<code>vpnServerPublicConfig.vpnPort</code>	string	default <code>51820</code>
<code>vpnServerPublicConfig.vpnSshCredentialsRef</code>	secret ref	
<code>softwareConfig.kubernetesVersion</code>	string	
<code>softwareConfig.imagePrepulls[]</code>	<code>{image, nodeTarget}</code>	
<code>softwareConfig.imagePullSecretRef</code>	secret ref	
<code>softwareConfig.cnlabRuntime</code>	object	same shape/defaults as <code>RemoteCluster</code> 's

Status:

Field	Meaning
<code>usedIPAddresses[]</code>	VPN IPs already allocated in this cluster
<code>clusterJoinCommand</code>	Join command for workers
<code>joinTokenRefreshedAt</code>	Last token refresh timestamp
<code>vpnPeers[]</code>	<code>{nodeName, publicKey, vpnIP}</code>
<code>kubeconfig</code>	Base64 admin kubeconfig for this cluster

Not supported today (shown in some sample files, but silently ignored): `softwareConfig.nvidiaDriverVersion` , `softwareConfig.nvidiaContainerToolkitVersion` , `softwareConfig.k8sDevicePluginVersion` .

10.4 Common `kubectl` commands

Command	Effect
<code>kubectl get remotecluster -w</code>	Watch a cluster's provisioning progress
<code>kubectl get nodeprovision -w</code>	Watch a node's provisioning progress
<code>kubectl patch remotecluster <name> --subresource=status -type=merge -p '{"status":{"provisionRetryCount":0}}'</code>	Reset a stuck <code>RemoteCluster</code> after fixing the underlying issue
<code>kubectl patch remotecluster <name> --subresource=status -type=merge -p '{"status":{"cnlabSyncRetryCount":0}}'</code>	Reset a stuck registry-credential sync
<code>kubectl patch nodeprovision <name> --subresource=status -type=merge -p '{"status":{"provisionRetryCount":0}}'</code>	Reset a stuck <code>NodeProvision</code>
<code>kubectl delete remotecluster <name></code>	Tear a node/cluster back down cleanly
<code>kubectl delete nodeprovision <name></code>	Remove a node cleanly

10.5 Install/upgrade commands

Command	Effect
<code>make install</code> / <code>make uninstall</code>	Apply/remove the CRDs
<code>make deploy IMG=<image></code> / <code>make undeploy</code>	Apply/remove the controller (pointing at an already-built image)

10.6 Where to find example manifests

Path	Contents
<code>config/samples/</code>	Example <code>RemoteCluster</code> / <code>NodeProvision</code> / <code>NodeProvisionNetConfig</code> resources — see §6
<code>docs/wireguard-setup-bundle/WIREGUARD_SETUP.md</code>	WireGuard VPN server/client setup guide
<code>README.md</code>	Top-level project overview

10.7 Useful links

- Project README: [../README.md](#)
- WireGuard setup guide: [wireguard-setup-bundle/WIREGUARD_SETUP.md](#)
- Nephio/Porch documentation (for GitOps-based platform deployment): <https://docs.nephio.org/>

This guide covers day-to-day use of the controllers — creating and managing clusters and nodes. It intentionally leaves out anything about building the controller image or its source code.